Interpolation for the simply-typed λ-calculus

Meven Lennon-Bertrand Alexis Saurin

November 26, 2025

Rewriting Čubrić [Čub94] in a modern type-theoretic language.

1 Context

Čubrić [Čub94] proves interpolation for the internal language of biCCC, that is, STLC with at least pair, unit, sum, and empty types, all with η-laws (including for positive connectives!), plus arbitrary base types, constants, and equations.

He really needs commuting conversions for $+$ and $\bot$, apparently because of some subformula property. I wonder how this looks like in our setting...

2 Definitions

2.1 λ-calculus

Definition We work with STLC, which we take to always have unit, product and function types (*i.e.* this is the internal language of CCC). We moreover work with respect to a theory Θ.

Definition 1 – Language

A *language* Λ consists of:

- a set of base types $\mathscr{B} \in \mathbf{Set}$;
- a family of typed constants indexed by types $\mathscr{C} \in \mathrm{Ty}_{\mathscr{B}} \to \mathbf{Set}$;

In the above, $\mathrm{Ty}_{\mathscr{B}}$ is the set of types generated by $\mathscr{B}$ which will be defined in Fig. 1. If $\Lambda = (\mathscr{B}, \mathscr{C})$ we write Ty_{Λ} for $\mathrm{Ty}_{\mathscr{B}}$, and we write $\emptyset$ for the empty language, *i.e.* that where the sets of base types and constants are empty.

Technically, we should interleave the definition of types generated by a set of base types with that of a language, but that's annoying (and becomes impossible anyway once you move to dependent type theory, see *e.g.* Bezem et al. [Bez+21] for the issue and an elegant solution).

Definition 2 – Theory

A *theory* Θ consists of:

- a language Λ;

$\boxed{\vdash \Gamma}$ Contexts $\Gamma, \Delta, \cdots \in \mathrm{Ctx}_\Lambda$

$$\textsc{Emp}\ \frac{}{\vdash \cdot} \qquad \textsc{Ext}\ \frac{\vdash \Gamma \qquad \vdash A}{\vdash \Gamma.(x\colon A)}$$

$\boxed{\vdash A}$ Types $A, B, \cdots \in \mathrm{Ty}_\Lambda$

$$\textsc{Base}\ \frac{b \in \mathscr{B}}{\vdash b} \qquad \textsc{Unit}\ \frac{}{\vdash \top} \qquad \textsc{Prod}\ \frac{\vdash A \qquad \vdash B}{\vdash A \times B} \qquad \textsc{Fun}\ \frac{\vdash A \qquad \vdash B}{\vdash A \to B}$$

$\boxed{\Gamma \vdash t : A}$ Terms $t, u, \cdots \in \mathrm{Tm}_\Lambda(A)$

$$\textsc{Cst}\ \frac{c \in \mathscr{C}(A)}{\Gamma \vdash c : A} \qquad \textsc{Var}\ \frac{(x\colon A) \in \Gamma}{\Gamma \vdash x : A} \qquad \textsc{Star}\ \frac{}{\Gamma \vdash * : \top}$$

$$\textsc{Pair}\ \frac{\Gamma \vdash t : A \qquad \Gamma \vdash u : B}{\Gamma \vdash (t, u) : A \times B} \qquad \textsc{Proj1}\ \frac{\Gamma \vdash p : A \times B}{\Gamma \vdash p.1 : A} \qquad \textsc{Proj2}\ \frac{\Gamma \vdash p : A \times B}{\Gamma \vdash p.2 : B}$$

$$\textsc{Lam}\ \frac{\Gamma.(x\colon A) \vdash t : B}{\Gamma \vdash \lambda\, x.t : A \to B} \qquad \textsc{App}\ \frac{\Gamma \vdash t : A \to B \qquad \Gamma \vdash u : A}{\Gamma \vdash t\ u : B}$$

Figure 1: Types and terms of the simply-typed λ-calculus (with respect to a language $\Lambda = (\mathscr{B}, \mathscr{C})$)

- a set of equations $\mathscr{E} : (A\colon \mathrm{Ty}_\Lambda) \to \mathscr{P}\,(\mathrm{Tm}_\Lambda(A)^2)$.

$\mathrm{Tm}_\Lambda(A)$ is the set of terms of type A generated by the language Λ, to be defined in Fig. 1.
If $\Theta = (\Lambda, \mathscr{E})$ we write Ty_Θ for Ty_Λ, and similarly for Tm.

The definition of types and terms is given in Fig. 1. MLB ▸*Technically, we should specify the substitution calculus...*◂

The equational theory is in Fig. 2. We omit congruence equations.

Extra operations

Definition 3 – Context partition

The context partition relation is defined as a (partial) relation $\Gamma = \Gamma_1 \cup \Gamma_2$ by

$$\frac{}{\cdot = \cdot \cup \cdot} \qquad \frac{\Gamma = \Gamma_1 \cup \Gamma_2}{\Gamma.(x\colon A) = \Gamma_1.(x\colon A) \cup \Gamma_2} \qquad \frac{\Gamma = \Gamma_1 \cup \Gamma_2}{\Gamma.(x\colon A) = \Gamma_1 \cup \Gamma_2.(x\colon A)}$$

Definition 4 – Reduction

Reduction is written $\rightsquigarrow$. MLB ▸*What sort of reduction is this?*◂

$\boxed{\Gamma \vdash t \equiv u : A}$ Equality of terms

$$\textsc{Refl}\ \frac{\Gamma \vdash t : A}{\Gamma \vdash t \equiv t : A} \qquad \textsc{Sym}\ \frac{\Gamma \vdash t \equiv u : A}{\Gamma \vdash u \equiv t : A} \qquad \textsc{Refl}\ \frac{\Gamma \vdash t \equiv u : A \qquad \Gamma \vdash t \equiv v : A}{\Gamma \vdash t \equiv v : A}$$

congruence/substitution rule

$$\textsc{Eq}\ \frac{(t, u) \in \mathscr{E}(A)}{\Gamma \vdash t \equiv u : A} \qquad \textsc{Unit}\eta\ \frac{\Gamma \vdash t : \top}{\Gamma \vdash t \equiv * : \top}$$

$$\textsc{Pair}\beta\ \frac{\Gamma \vdash t : A \qquad \Gamma \vdash u : B}{\Gamma \vdash (t, u)._1 \equiv t : A \qquad \Gamma \vdash (t, u)._2 \equiv u : A} \qquad \textsc{Pair}\eta\ \frac{\Gamma \vdash p : A \times B}{\Gamma \vdash p \equiv (p._1, p._2) : A \times B}$$

$$\textsc{Fun}\beta\ \frac{\Gamma, x\colon A \vdash t : B \qquad \Gamma \vdash u : A}{\Gamma \vdash (\lambda\, x.t)\ u \equiv t[x := u] : A} \qquad \textsc{Fun}\eta\ \frac{\Gamma \vdash f : A \to B}{\Gamma \vdash f \equiv \lambda\, x.(f\ x) : A \to B}$$

Figure 2: Equations of the simply-typed λ-calculus (with respect to a theory $\Theta = (\Lambda, \mathscr{E})$)

2.2 Bidirectional typing

We can characterise those terms which have a nice "information flow" by using bidirectional typing. Incidentally, this characterises normal/neutral forms – a very deep mystery, but a useful one. Contexts and types are the same as in Fig. 1 above, the term judgments are in Fig. 3.

There is an embedding of bidirectional terms into undirected ones, which we call erasure and write

$$|\cdot| : \mathrm{Nf}_\Lambda(A) \to \mathrm{Tm}_\Lambda(A)$$
$$|\cdot| : \mathrm{Ne}_\Lambda(A) \to \mathrm{Tm}_\Lambda(A)$$

Both are entirely structural, up to $|\underline{t}| = |t|$. We generally leave them implicit.

Theorem 5 – Normalisation

If Σ is a theory with no equations, given a term $t \in \mathrm{Tm}_\Sigma(A)$, there exists a unique $n \in \mathrm{Nf}_\Sigma(A)$ such that $t = |n|$.

2.3 Evaluation context

Another way to capture neutrals, which looks a bit closer to what is happening in Čubrić's [Čub94] proof, is to use *evaluation stacks*, see Fig. 4. A stack represents a succession of elimination forms. Any neutral can be decomposed into a variable plugged into such a stack, which captures the succession of eliminations which compose the neutral. The important difference is that stacks are "inside out", *i.e.* the top of the stack corresponds to the innermost elimination of the neutral. This seems to be closer to what happens in Čubrić [Čub94].

$\boxed{\Gamma \vdash t \triangleleft A}$ Checking/normal terms $t, u, \cdots \in \mathrm{Nf}_\Lambda(\Gamma, A)$

$$\textsc{Star}\ \frac{}{\Gamma \vdash * \triangleleft \top} \qquad \textsc{Pair}\ \frac{\Gamma \vdash t \triangleleft A \qquad \Gamma \vdash u \triangleleft B}{\Gamma \vdash (t, u) \triangleleft A \times B} \qquad \textsc{Lam}\ \frac{\Gamma, x\colon A \vdash t \triangleleft B}{\Gamma \vdash \lambda\, x.t \triangleleft A \to B}$$

$$\textsc{Demote}\ \frac{\Gamma \vdash t \triangleright A \qquad A = B}{\Gamma \vdash \underline{t} \triangleleft B}$$

$\boxed{\Gamma \vdash t \triangleright A}$ Infering/neutral terms $t, u, \cdots \in \mathrm{Ne}_\Lambda(\Gamma, A)$

$$\textsc{Cst}\ \frac{c \in \mathscr{C}(A)}{\Gamma \vdash c \triangleright A}\ \boxed{\textsc{mlb}}\ \blacktriangleright \textit{Is this infering or checking?} \blacktriangleleft \qquad \textsc{Var}\ \frac{(x\colon A) \in \Gamma}{\Gamma \vdash x \triangleright A}$$

$$\textsc{Proj1}\ \frac{\Gamma \vdash p \triangleright A \times B}{\Gamma \vdash \pi_1(p) \triangleright A} \qquad \textsc{Proj2}\ \frac{\Gamma \vdash p \triangleright A \times B}{\Gamma \vdash \pi_2(p) \triangleright B} \qquad \textsc{App}\ \frac{\Gamma \vdash t \triangleright A \to B \qquad \Gamma \vdash u \triangleleft A}{\Gamma \vdash t\ u \triangleright B}$$

Figure 3: Bidirectional typing for the simply-typed λ-calculus (with respect to a language $\Lambda = (\mathscr{B}, \mathscr{C})$)

$\boxed{\Gamma \mid A \vdash \pi \triangleright B}$ Stacks $\pi, \cdots \in \mathrm{Sta}_\Lambda(\Gamma, A, B)$

$$\textsc{Nil}\ \frac{}{\Gamma \mid T \vdash [] \triangleright T} \qquad \textsc{Proji}\ \frac{\Gamma \mid A_i \vdash \pi \triangleright T}{\Gamma \mid A_1 \times A_2 \vdash \square.{}_i :: \pi \triangleright T} \qquad \textsc{App}\ \frac{\Gamma \mid B \vdash \pi \triangleright T \qquad \Gamma \vdash u \triangleleft A}{\Gamma \mid A \to B \vdash \square\ u :: \pi \triangleright T}$$

Figure 4: Evaluation stacks for the simply-typed λ-calculus

Definition 6 – Stack operations

Given a stack and a neutral, we can always plug the latter into the former:

$$\begin{array}{rcl} \_[\_] : \mathrm{Sta}(\Gamma, A, B) \to \mathrm{Ne}(\Gamma, A) & \to & \mathrm{Ne}(\Gamma, B) \\ [][t] & \coloneqq & t \\ (\square.i :: \pi)[t] & \coloneqq & \pi[t.i] \\ (\square\ u :: \pi)[t] & \coloneqq & \pi[t\ u] \end{array}$$

Conversely, any neutral can be (uniquely) decomposed into a form $\pi[x]$.

3 Interpolation

Definition 7 – Polarities

Let $A \in \mathrm{Ty}_{\mathscr{B}}$. We define $A^+ \subseteq \mathscr{B}$ (resp. $A^- \subseteq \mathscr{B}$) by induction:

$$\begin{aligned} b^+ &\coloneqq \{b\} \\ b^- &\coloneqq \emptyset \\ (A \times B)^p &\coloneqq A^p \cup B^p \\ (A \to B)^p &\coloneqq A^{-p} \cup B^p \\ \top^p &\coloneqq \emptyset \end{aligned}$$

This is lifted to contexts as $(\Gamma.(x\colon A))^p \coloneqq \Gamma^p \cup A^p$.

Main theorem The first version of interpolation works with constant-free theories, *i.e.* those of the form $((\mathscr{B}, \emptyset), \emptyset)$.

Theorem 8 – Interpolation – constant-free

Let Σ be a constant-free theory, and assume a partitioned context $\Gamma = \Gamma_s \cup \Gamma_t$.

Given any normal term $t \in \mathrm{Nf}_\Sigma(\Gamma, T)$, there is a type $M \in \mathrm{Ty}_\Sigma$ and terms $l \in \mathrm{Tm}_\Sigma(\Gamma_s, M)$ and $r \in \mathrm{Tm}_\Sigma(\Gamma_t.(x\colon M), T)$ such that

- $\Gamma \vdash r[x \coloneqq l] \equiv t : T$;
- $M^+ \subseteq \Gamma_s^+ \cap (\Gamma_t^- \cup T^+)$ and $M^- \subseteq \Gamma_s^- \cap (\Gamma_t^+ \cup T^-)$.

Moreover, given any neutral term $t \in \mathrm{Ne}_\Sigma(\Gamma, T)$, there is a type $M \in \mathrm{Ty}_\Sigma$ such that either:

1. "T is a source type", *i.e.* we have terms $l \in \mathrm{Tm}_\Sigma(\Gamma_t, M)$ and $r \in \mathrm{Tm}_\Sigma(\Gamma_s.(x\colon M), T)$ such that
   - $\Gamma \vdash r[x \coloneqq l] \equiv t : T$,
   - $T^+ \subseteq \Gamma_s^+$ and $T^- \subseteq \Gamma_s^-$,
   - $M^+ \subseteq \Gamma_s^+ \cap (\Gamma_t^- \cup T^+)$ and $M^- \subseteq \Gamma_s^- \cap (\Gamma_t^+ \cup T^-)$;
2. or "T is a target type", *i.e.* we have terms $l \in \mathrm{Tm}_\Sigma(\Gamma_s, M)$ and $r \in \mathrm{Tm}_\Sigma(\Gamma_t.(x\colon M), T)$ such that
   - $\Gamma \vdash r[x \coloneqq l] \equiv t : T$,
   - $T^+ \subseteq \Gamma_t^+$ and $T^- \subseteq \Gamma_t^-$,
   - $M^+ \subseteq \Gamma_s^+ \cap (\Gamma_t^- \cup T^+)$ and $M^- \subseteq \Gamma_s^- \cap (\Gamma_t^+ \cup T^-)$.

Proof

By induction on the definition of Ne / Nf.
The Nf cases are rather direct.
Star: T is $\top$, so we can take $M = \top$, and $l = r = \star$.
Pair: T is $T_1 \times T_2$, t is (t_1, t_2). By induction, we obtain $l_i \in \mathrm{Tm}(\Gamma_s, M_i)$ and $r_i \in \mathrm{Tm}(\Gamma_t.(x\colon M_i), T_i)$. We can then take $M := M_1 \times M_2$ and

$$\begin{array}{rrll} \Gamma_s \vdash & l := & (l_1, l_2) & : M_1 \times M_2 \\ \Gamma_t.(x\colon M_1 \times M_2) \vdash & r := & (r_1[y := y.1], r_2[y := y.2]) & : T_1 \times T_2 \end{array}$$

Lam: We extend the *target* context with the extra variable (this is why we need this more complicated formulation with a context split!) and by induction hypothesis, we obtain $l' \in \mathrm{Tm}(\Gamma_s, M)$ and $r' \in \mathrm{Tm}(\Gamma_t.(x\colon M).(y\colon A))$, and we can take

$$\begin{array}{rrll} \Gamma_s \vdash & l := & l' & : M \\ \Gamma_t.(x\colon M) \vdash & r := & \lambda\, y.r' & : A \to B \end{array}$$

The Ne cases are more interesting. For each rule, we have two subcases, depending on the source/target induction hypothesis for the main sub-neutral, or on whether the variable in is the source or target context.
Var/1: We have $(x : A) \in \Gamma_s$. Then we take $M := \top$

$$\begin{array}{rrll} \Gamma_t \vdash & l := & \star & : M \\ \Gamma_s.(\_\colon M) \vdash & r := & x & : A \end{array}$$

Var/2: We have $(x : A) \in \Gamma_t$. Then we take $M := \top$

$$\begin{array}{rrll} \Gamma_s \vdash & l := & \star & : M \\ \Gamma_t.(\_\colon M) \vdash & r := & x & : A \end{array}$$

Demote/1: By induction hypothesis, we get that A is a "source type", $l \in \mathrm{Tm}_\Sigma(\Gamma_t, M)$ and $r \in \mathrm{Tm}_\Sigma(\Gamma_s.(x\colon M), A)$. But because of the constraint of the Demote rule, it must be the case that $A = B$, in other words these type really are in the common language of Γ_s and Γ_t. Thus, we take $M' := M \to A$ and

$$\begin{array}{rrll} \Gamma_s \vdash & l' := & \lambda\, x.r & : M \to A \\ \Gamma_t.(z\colon M) \vdash & r' := & z\, l & : A \end{array}$$

Demote/2: In this case we keep the exact same interpolating type and interpolants.
App/1: We are in the case of an application $t\ u$, with the induction hypothesis on t giving us some $A \to B$ with vocabulary in Γ_s and M_t, r_t and l_t such that $t = r_t[x := l_t]$. Because A has its vocabulary in Γ_s, by induction hypothesis we can interpolate u with a "reversed" colouring on Γ, *i.e.* inverting the roles of Γ_s and Γ_t. This gives us M_u, r_u and l_u such that $u = r_u[x := l_u]$. Then we take $M := M_t \times M_u$, and

$$\begin{array}{rrll} \Gamma_t \vdash & l := & (l_t, l_u) & : M_t \times M_u \\ \Gamma_t.(z\colon M_t \times M_u) \vdash & r := & r_t[x := z.1]\ r_u[x := z.2] & : B \end{array}$$

App/2: This time, the induction hypothesis on t gives us some $A \to B$ with vocabulary in Γ_s and M_t, r_t and l_t such that $t = r_t[x := l_t]$. We can directly use the induction hypothesis on u, to get M_u, r_u and l_u such that $u = r_u[x := l_u]$. Then we take again $M := M_t \times M_u$, and

$$\begin{array}{rrll} \Gamma_s \vdash & l := & (l_t, l_u) & : M_t \times M_u \\ \Gamma_t.(z\colon M_t \times M_u) \vdash & r := & r_t[x := z.1]\ r_u[x := z.2] & : B \end{array}$$

PROJi: Here in all cases we keep the same interpolating type and left interpolant, and simply post-compose the right interpolant with the relevant projection.

It seems we do not quite construct the same interpolant as Čubrić. Indeed, in the case of a stack of applications on a source variable of type $A_1 \to \ldots \to A_n \to B$, we will construct an interpolating type $(M_1 \times \cdots \times M_n) \to B$, while Čubrić will rather construct $M_1 \to \ldots \to M_n \to B$.

References

[Bez+21] M. Bezem, T. Coquand, P. Dybjer, and M. Escardó. "On generalized algebraic theories and categories with families". In: *Mathematical Structures in Computer Science* 9 (2021), pp. 1006–1023. DOI: 10.1017/S0960129521000268. URL: https://www.cambridge.org/core/product/02459F7C89C72EEA9A1BC296F17B920C (cit. on p. 1).

[Čub94] D. Čubrić. "Interpolation property for bicartesian closed categories". In: *Archive for Mathematical Logic* 4 (Aug. 1994), pp. 291–319. DOI: 10.1007/bf01270628 (cit. on pp. 1, 3, 7).